Using RNN Classification to solve Simple Substitution Ciphers

A Final Report by David Hamilton, Anosh Askari, and Varma Indukuri

Abstract

Substitution ciphers represent one of the earliest cryptographic techniques, with their origins tracing back nearly as far as the inception of writing itself. The earliest known descriptions of methods to crack these ciphers date to approximately 850 CE. Traditional decryption methods largely rely on frequency analysis and the recognition of letter patterns—techniques that, while historically effective, present substantial challenges when adapted to modern computational algorithms. This project addresses these challenges by employing advanced machine learning technologies, specifically bidirectional gated recurrent units (GRU) and long short-term memory (LSTM) networks with a search algorithm that maximizes plaintext probability. These recurrent neural networks (RNNs) are designed to predict unknown letters in texts ranging from fully ciphered to partially deciphered, thus enhancing the decryption process. The primary innovation of our approach lies in the integration of RNNs with a dynamic search algorithm. This algorithm methodically explores the most probable guesses for each unknown letter until the entire ciphered input is deciphered. By modifying the Brown Corpus from the Natural Language Toolkit (NLTK) to train our model with our approach, the final models were able to classify both different steps in the decryption process ciphered inputs with a high degree of accuracy, achieving validation and test accuracies close to and above 89.5%. To test decrypting the cipher completely, we used test cases sourced from https://api.razzlepuzzles.com/cryptogram, which is the website that inspired this project. As they were not contained at all in the Brown Corpus, they presented a unique challenge to our models. The performance of the models on these external ciphers was mixed; some texts were fully deciphered, while others were only partially solved, with one or two letters correctly identified. This variance in success rates can be attributed to several factors, including the length of the input sentences and the frequency of uncommon letters within the ciphers. Our observations indicated that longer sentences typically yielded more successful decryption outcomes. Conversely, ciphers composed of uncommon letters (such as 'z', 'q', 'j', 'x') and of shorter length tended to be more challenging for the models, suggesting a potential area for further improvement. Recommendations for further research is to incorporate a greater number of sentences that include less common letters into the training set, and to refine the algorithm in which the decryption process is approached could significantly improve the model's ability to correctly classify uncommon letters more accurately.

Introduction

The significance of breaking substitution ciphers lies in the difficulty in approaching the problem. A homophonic substitution cipher is where one letter of the plaintext is represented by a letter from a shuffled alphabet, creating a cipher text. Traditional methods to crack these ciphers rely heavily on frequency analysis and recognizing letter patterns in the language, which can be time-consuming and are common puzzles found in newspapers or online for people to do. The difficulty in teaching an AI to solve the cipher is because the AI needs to be able to find context and patterns in text to correctly classify a letter. Our research pivots towards a modern computational approach, utilizing the robust capabilities of recurrent neural networks (RNNs) to classify letters in coordination with a search algorithm to automate the deciphering process.

Our project leverages bi-directional GRUs and LSTMs to tackle the challenges presented by substitution ciphers. These RNN architectures are particularly adept at processing sequences of data, making them ideal for understanding the context and dependencies within encrypted texts. By training our models on the Brown Corpus, provided by the Natural Language Toolkit (NLTK), the models learn to analyze letter patterns within word to predict unknown letters in texts that range from completely to partially ciphered. The use of bi-directional networks allows our models to capture information from both past and future contexts within the encrypted text, significantly enhancing the accuracy of predictions.

Recovering the plaintext from the ciphertext can be computationally expensive because of the number of possible permutations to explore what the next successor state is in the decoding process. Our approach optimizes this by reordering the input text by the number of unique unciphered letters remaining(example shown later), and then guessing the most common unciphered letter in the word ordered first. We use bidirectional LSTM and GRUs for classification, so that ciphertexts of varying lengths can be input. When deciphering the whole ciphertext from start to finish, we use an Expectimax-Dijkstra search hybrid, where we search for the most probable plaintext given the ciphertext.

Related existing work uses an LSTM exclusively along with different modifications of beam search[1]. One modification uses an N-gram of 6 on the input text, and another modification uses a frequency matching heuristic. The LSTM they used was a multiplicative LSTM with 4096 units. For comparison, our largest model has a bidirectional LSTM with 512 units.

Dataset Statistics

The dataset we used was the Brown Corpus and was accessed through NLTK. After removing all characters except letters and apostrophes, we also removed sentences that had lengths of unique words that were above a length of 250 or below a length of 10. For example, the sentence “I want to go to dinner” would have had a length of 19, because the second “to” would have been removed when counting the length. After this processing and removing sentences of

improper lengths, we shuffled the corpus and then did a training, validation, and testing split of 80-10-10. This resulted in 43,689 sentences in the training set; 5,461 in the validation set; and 5462 in the testing set. Using the sets, we generated the ciphers for classification. This was done by using a unique seed to generate 3 cipher keys per sentence(each seed was recorded in a set to confirm no cipher key was duplicated), and then each step in the decipherment process was added to a new set for training, validating, and testing the model depending on its origin. A capital letter represents the ciphertext, a lowercase letter represents plaintext, and the underscore is the value next to be classified. This process is described below.

Provided Sentence: i think it's a good day for a walk don't you

Step 1 - Encrypt the plaintext

Result: G CZGST GC'B M VHHY YMP EHO M WMKT YHS'C PHX

Step 2 - Remove duplicated words

Result: G CZGST GC'B M VHHY YMP EHO WMKT YHS'C PHX

Step 3 - Sort word order by number of unique unciphered letters

Result: G M GC'B VHHY YMP EHO PHX WMKT YHS'C CZGST

Step 4 - Replace most common unciphered letter in first word that contains unciphered letters and with an underscore and add it to the corpus

Result: _ M _C'B VHHY YMP EHO PHX WMKT YHS'C CZ_ST,i

Step 5 - Replace underscore with class and resort word order by number of unique unciphered letters

Result: i M iC'B VHHY YMP EHO PHX WMKT YHS'C CZiST

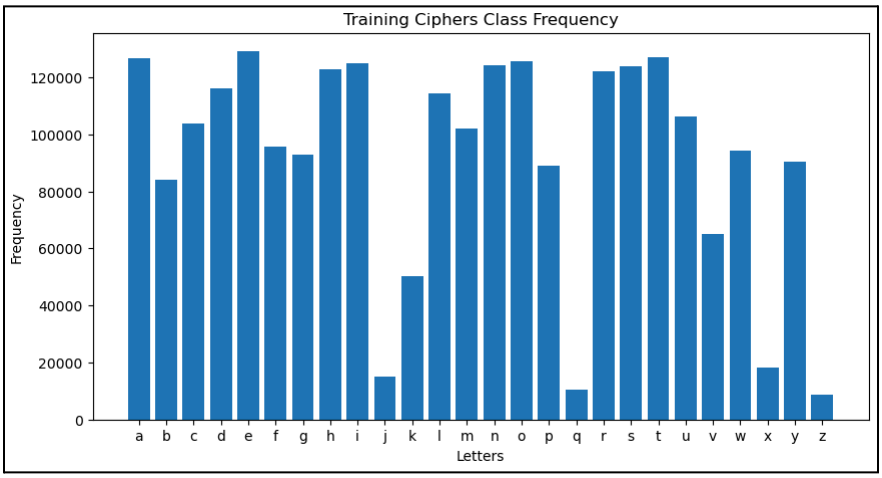
Step 6 - Repeat Steps 4 and 5 until all steps have been generated

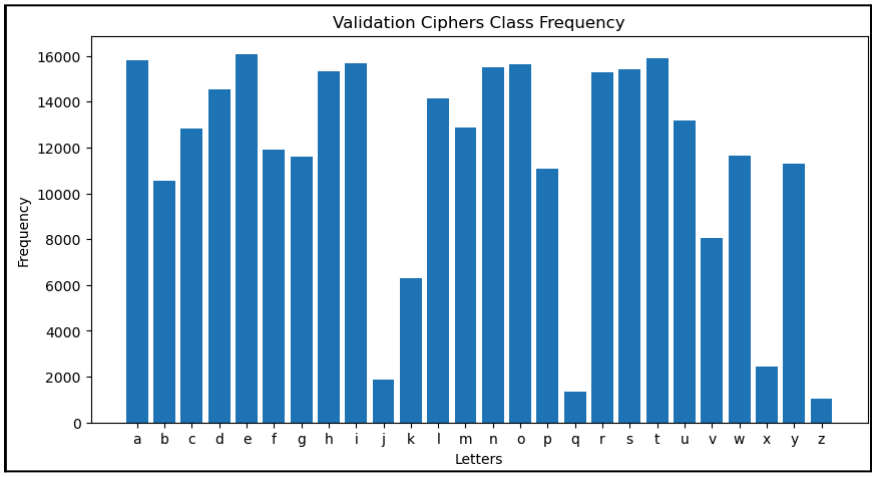
Resulting ciphers generated:

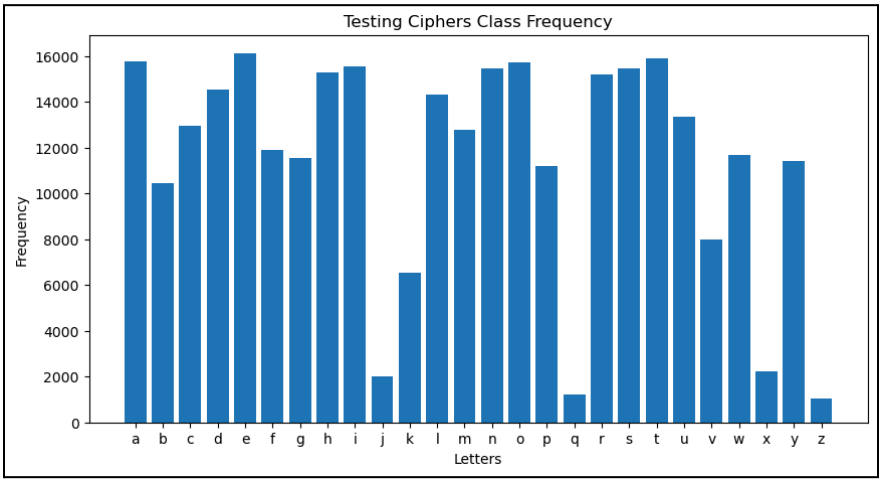
1. _ M _C'B VHHY YMP EHO PHX WMKT YHS'C CZ_ST,i
2. i _ iC'B VHHY Y_P EHO PHX W_KT YHS'C CZiST,a
3. i a i_'B YaP VHHY EHO PHX WaKT YHS'_ _ZiST,t
4. i a it'_ YaP VHHY EHO PHX WaKT YHS't tZiST,s
5. i a it's _aP VHH_ EHO PHX WaKT _HS't tZiST,d
6. i a it's da_ VHHd dHS't EHO _HX WaKT tZiST,y
7. i a it's day V__d d_S't y_X E_O WaKT tZiST,o
8. i a it's day _ood doS't yoX EoO WaKT tZiST,g
9. i a it's day good do_'t yoX EoO WaKT tZi_T,n
10. i a it's day good don't yo_ EoO tZinT WaKT,u
11. i a it's day good don't you _oO tZinT WaKT,f
12. i a it's day good don't you fo_ tZinT WaKT,r
13. i a it's day good don't you for tZin_ WaK_,k
14. i a it's day good don't you for t_ink WaKk,h
15. i a it's day good don't you for think _aKk,w
16. i a it's day good don't you for think wa_k,l

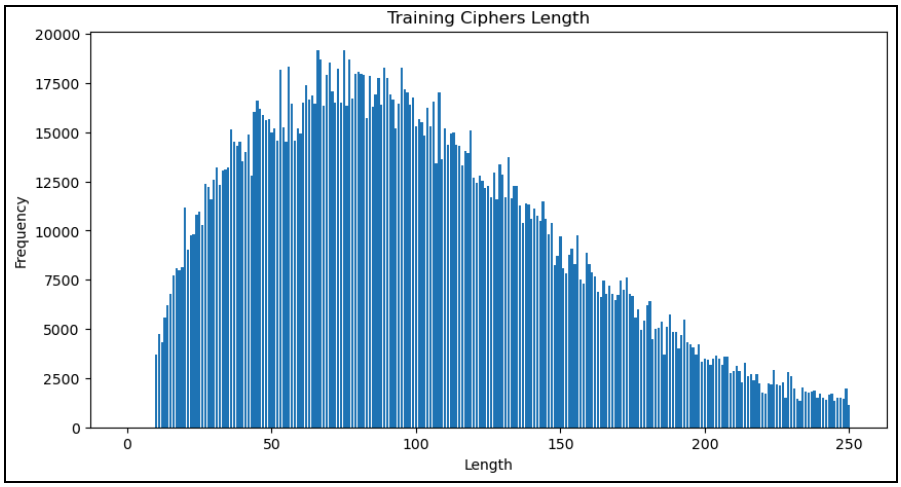
That one sentence generated 16 ciphers to be added to our new set, and because each sentence is encrypted 3 times, a total of 48 new sentences would have been made from that example. The formula for calculating the total number of ciphers generated per sentence is the

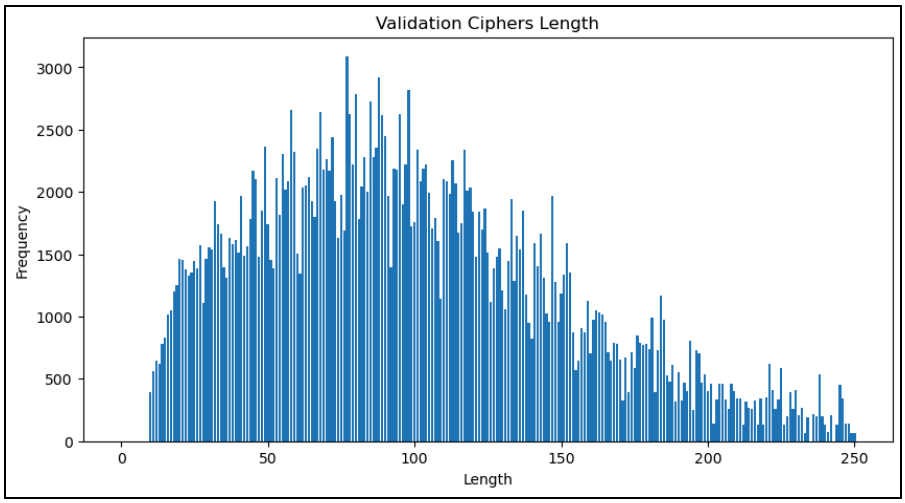
number of unique letters in the sentence times 3. This process increased our training set size to 2,381,889; validation set to size 297,324; and testing set size to 297,591. Shown below is the frequency counts of each class and a graph of the lengths of the ciphers in the sets.

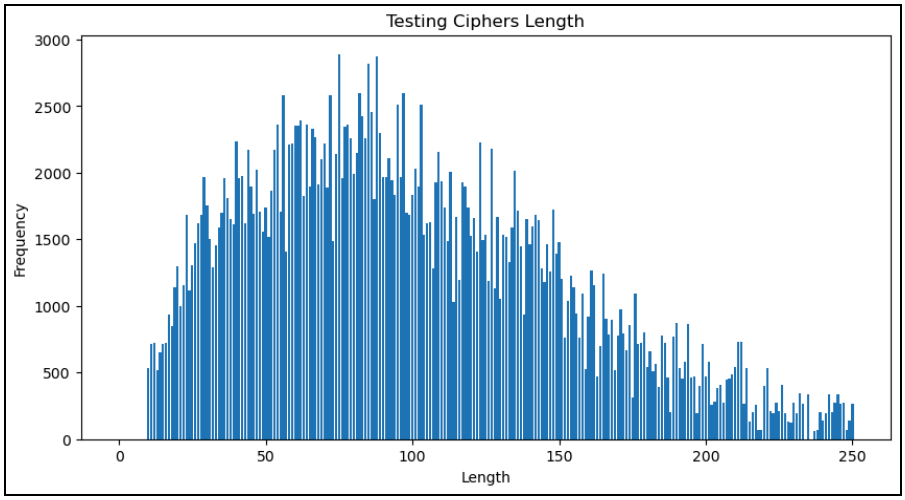




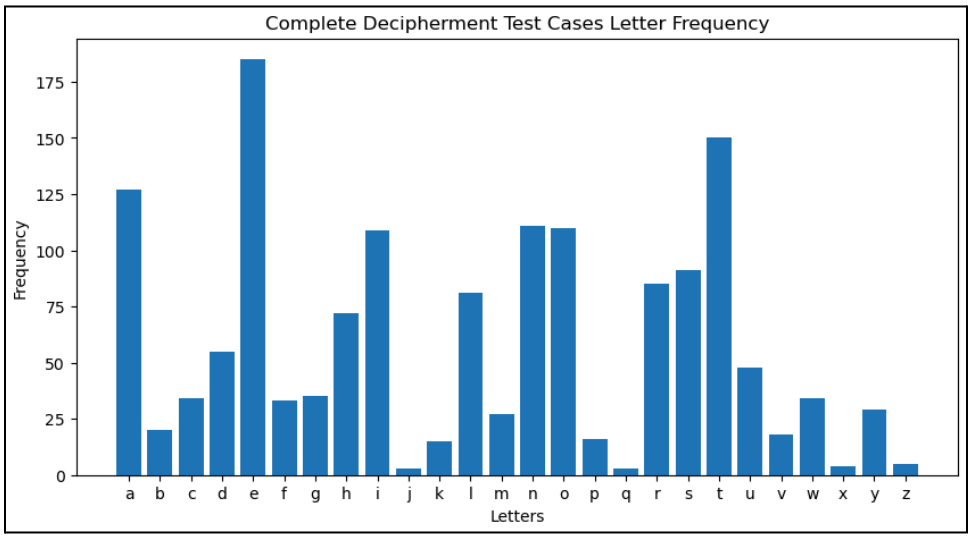



Training Ciphers Length
Frequency
Length
0
2500
5000
7500
10000
12500
15000
17500
20000
0
50
100
150
200
250


Validation Ciphers Length
Frequency
Length
0
500
1000
1500
2000
2500
3000
0
50
100
150
200
250


Testing Ciphers Length
Frequency
Length
0
500
1000
1500
2000
2500
3000
0
50
100
150
200
250

The test set used to evaluate the total decipherment are 22 cases not found in the Brown Corpus. All of them were found on the website https://api.razzlepuzzles.com/cryptogram with the exception of the example shown above, which was also included. The reason we did not use the test set for this was because many of the sentences that were made up of infrequent letters were made up of sentences that used all letters in the alphabet. Something that we discovered late in our project is that our models ended up working very poorly on cases that contained all letters of the alphabet, and for this reason, we ended up curating the test sentences. We chose this website to generate them because of how it inspired the project. Each of the were ciphered once, and then also the plaintext version of the sentence is also input capitalized to make the model think the plaintext is encrypted. The longest test-case post processing is 168 characters long, and the shortest is 17. All but five test cases fall between the lengths of 50 and 125.

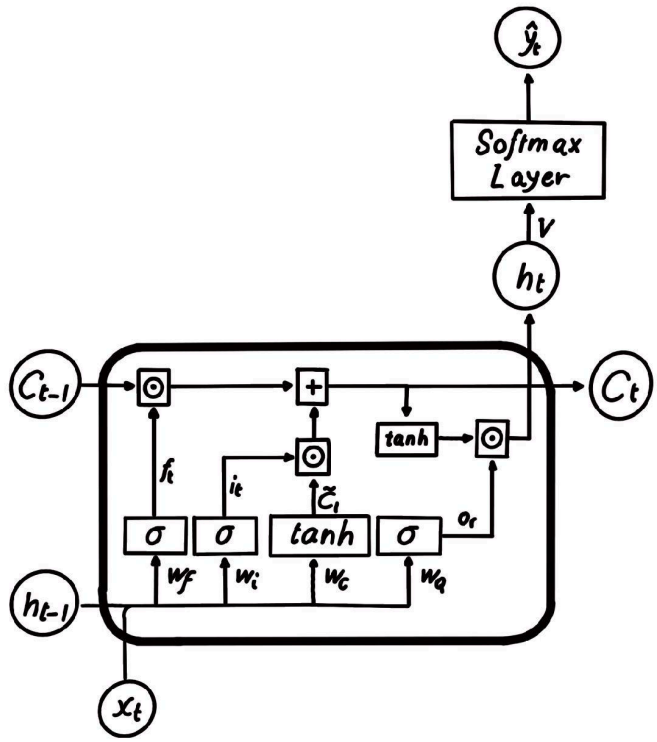


Model

We built our models by using keras.sequential. We started with an embedding layer for the character inputs, then our bidirectional LSTM/GRU layer, and then finally a dense layer of 27 that used categorical cross entropy. The dense layer was of size 27, because when it was size 26 the letter ‘a’ was never classified. This fixed the issue, and we moved on instead of wasting time troubleshooting it.

LSTM (Long Short-Term Memory):
LSTMs are designed to avoid the long-term dependency problem in traditional RNNs. They do this by maintaining a memory cell that can store information for long periods, controlled by structures called gates:
Forget Gate: Determines parts of the cell state to be thrown away.
Input Gate: Decides which new information is added to the cell state.

Output Gate: Determines what the next hidden state should be, which is used to predict the output.



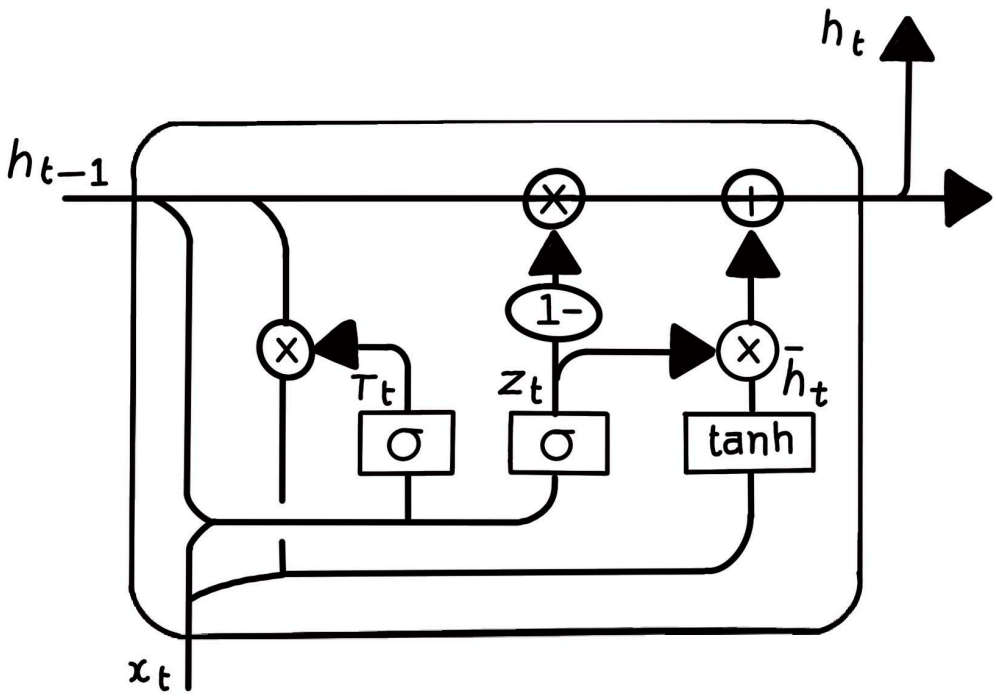
Each of these gates uses a combination of sigmoid activation (to control the binary decisions) and tanh activation (to create a vector of new candidate values).

$$
\begin{aligned}
i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\
f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\
o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\
\tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\
C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \\
h_t &= o_t * \tanh(C_t)
\end{aligned}
$$

GRU (Gated Recurrent Unit)

GRUs simplify the LSTM design by combining the forget and input gates into a single "update gate." They also merge the cell state and hidden state, resulting in fewer tensor operations.

$$
\begin{aligned}
z_t &= \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \\
r_t &= \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \\
\tilde{h}_t &= \tanh(W \cdot [r_t * h_{t-1}, x_t] + b_h) \\
h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t
\end{aligned}
$$



These models were configured using:

- an embedding layer which encodes character inputs into dense vectors, providing a richer representation of data.
- a bidirectional LSTM/GRU layer which processes input data in both forward and backward directions, enhancing the learning context.
- a dense Layer which outputs probabilities across 27 classes, ensuring all characters, including 'a', are properly classified.

The objective function used is categorical cross entropy which measures the difference between the predicted probabilities and the actual distribution, ideal for multi-class classification. It

ensures that the model learns to predict the probability distribution of the output classes correctly.

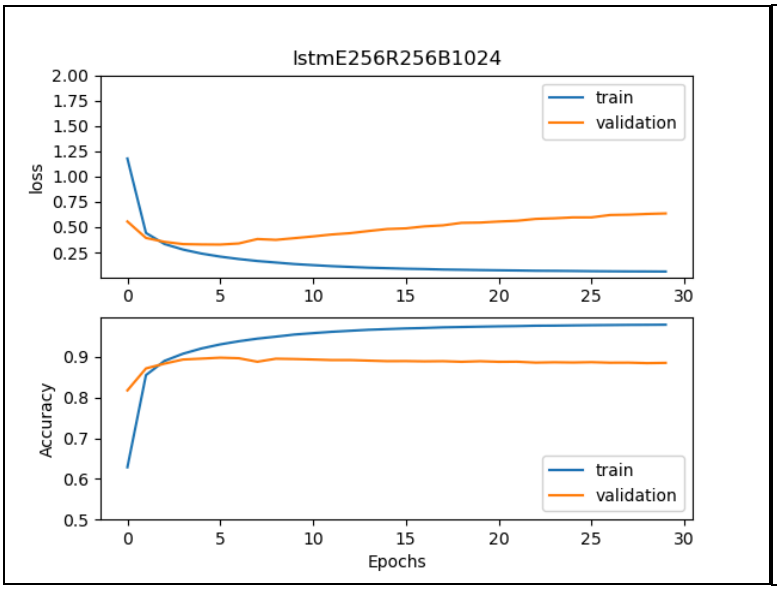
Our search algorithm is a hybrid of Dijkstra's search algorithm and Expectimax search. It is similar to Dijkstra's because all possible states are recorded and then the lowest cost state is searched next. The priority queue used contains the likelihood of that state so far, the partially deciphered cipher, and the cipher key as decoded so far. However, because we're using probability and the cost of each next state is unknown, the algorithm being used is similar to expectimax search, albeit without an opponent. The importance of keeping track of the key allows us to ignore already deciphered classes after each step. Our search algorithm also allows us to provide it with a key of known deciphered letters, which is handled carefully so that the algorithmic process to deciphering isn't messed up by starting the search with provided letters.

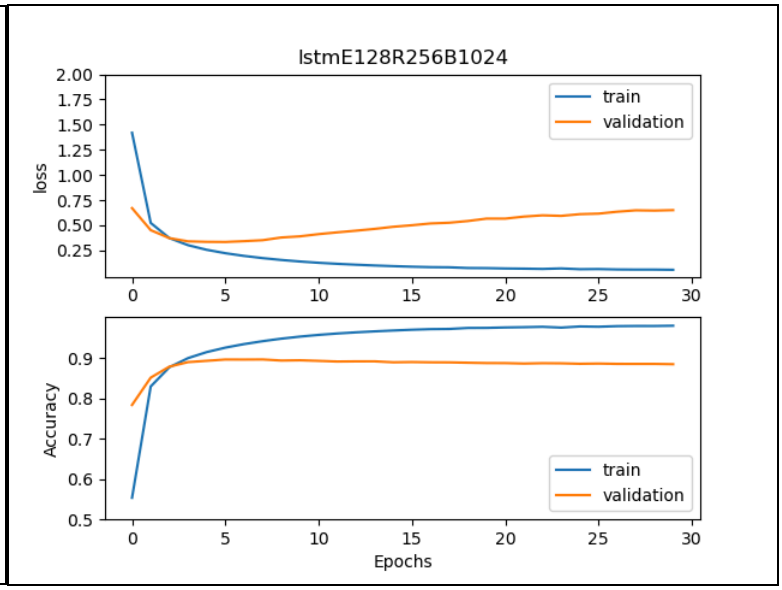
Experimental Setup

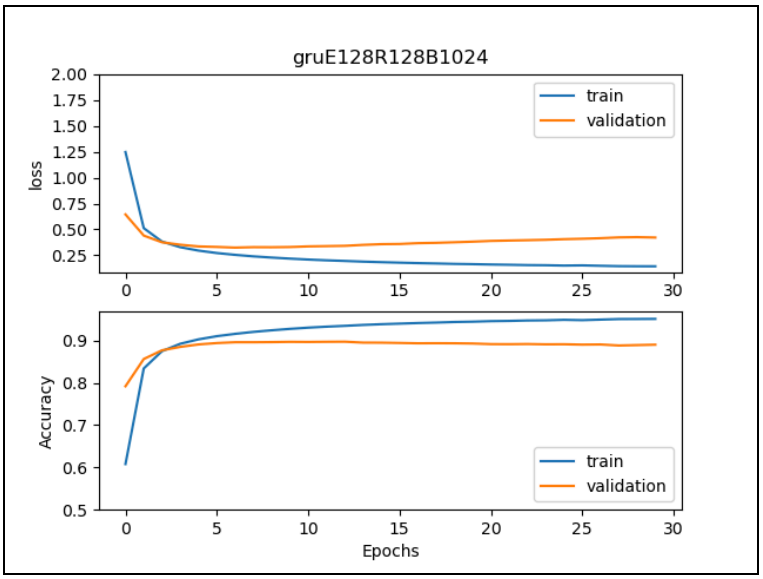
Because of the large number of samples, we used a standard batch size of 1024(which was close to the largest size the computer training the models could handle). We used the default Adam Optimizer, and then utilized grid search for optimal Embedding Dimensions of [64,128,256] in combination with RNN Units of sizes [64,128,256]. Because the RNN was bidirectional, the total number of RNN Units ended up being [128, 256, 512]. To monitor overtraining, we enabled early stopping by watching training loss, and then selected the checkpoint with the lowest validation loss for further analysis. Because we were searching through LSTM and GRU models, we ended up training 18 models for 30 Epochs each. The top models are bolded in the tables below, and have their loss and accuracies modeled.

GRU Optimization Results			
Embedding\RNN Units	64	128	256
64	Best Epoch:30 Training: (0.9012,0.2983) Validation: (0.8869,0.3472) Testing: (0.8842,0.3577)	Best Epoch:8 Training: (0.9082,0.2789) Validation: (0.8887,0.3474) Testing: (0.8842,0.3577)	Best Epoch:5 Training: (0.9168,0.2518) Validation: (0.8904,0.3454) Testing: (0.8876,0.3568)
128	Best Epoch:29 Training: (0.9029,0.2932) Validation: (0.8867,0.3495) Testing: (0.8846,0.3582)	**Best Epoch:7** **Training: (0.9156,0.2550)** **Validation: (0.8958,0.3248)** **Testing: (0.8930,0.3346)**	Best Epoch:5 Training: (0.9191,0.2438) Validation: (0.8923,0.3375) Testing: (0.8904,0.3462)
256	Best Epoch:30 Training: (0.9020,0.2957) Validation: (0.8868,0.3468) Testing: (0.8847,0.3580)	Best Epoch:11 Training: (0.9249,0.2259) Validation: (0.8930,0.3412) Testing: (0.8912,0.3498)	Best Epoch:4 Training: (0.9138,0.2609) Validation: (0.8917,0.3370) Testing: (0.8886,0.3484)

LSTM Optimization Results			
Embedding\RNN Units	64	128	256
64	Best Epoch:28 Training: (0.8861,0.3488) Validation: (0.8726,0.3927) Testing: (0.8696,0.4066)	Best Epoch:13 Training: (0.9036,0.2914) Validation: (0.8848,0.3561) Testing: (0.8826,0.3672)	Best Epoch:6 Training: (0.9191,0.2434) Validation: (0.8929,0.3414) Testing: (0.8905,0.3486)
128	Best Epoch:30 Training: (0.9000,0.3020) Validation: (0.8833,0.3643) Testing: (0.8808,0.3730)	Best Epoch:13 Training: (0.9156,0.2540) Validation: (0.8910,0.3409) Testing: (0.8886,0.3509)	**Best Epoch:6** **Training: (0.9261,0.2219)** **Validation: (0.8967,0.3335)** **Testing: (0.8930,0.3441)**
256	Best Epoch:29 Training: (0.8973,0.3112) Validation: (0.8814,0.3649) Testing: (0.8782,0.3786)	Best Epoch:10 Training: (0.9082,0.2762) Validation: (0.8889,0.3441) Testing: (0.8873,0.3510)	**Best Epoch:6** **Training: (0.9304,0.2087)** **Validation: (0.8917,0.3278)** **Testing: (0.8944,0.3385)**







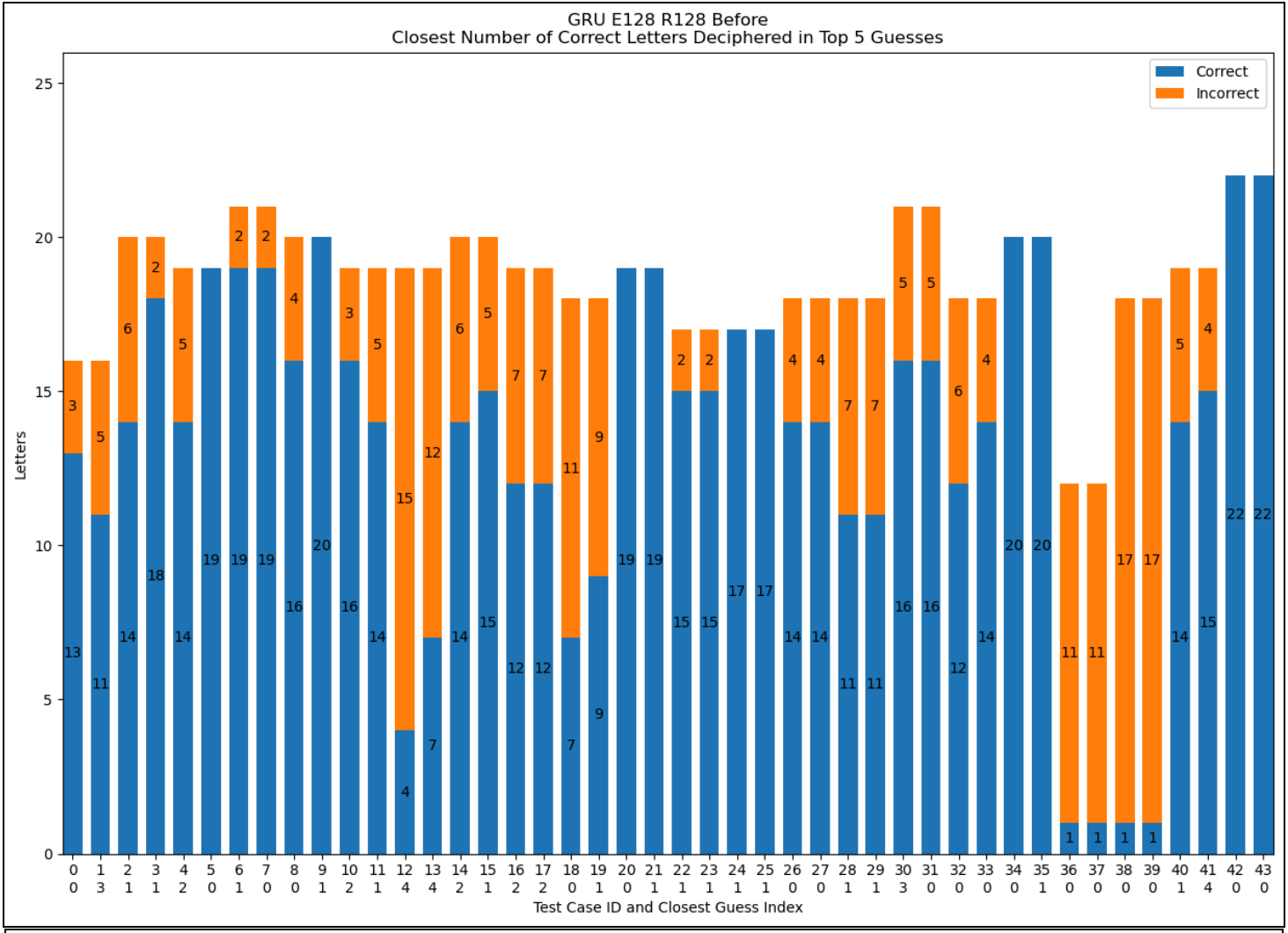
We then selected the three top models at their best checkpoint for further optimization. This was the GRU-E128-R128 at epoch 7, LSTM-E256-R-128 at epoch 6, and LSTM-E256-R256 at epoch 6. Using grid search again with a Learning Rate combination of [1E-7,5E-7,5E-6,1E-6] and Weight Decay of [0.95, 0.99, 0.9999] and 10 epochs, we decreased validation and testing loss and increased validation and testing accuracy on all three models. We tried optimizing without using weight decay, but without it, the models would stop improving after the first epoch. The models also only started decreasing loss once we started using learning rates equal to and below 1E-6. Once again we used early stopping that monitored training loss, and then selected the epochs with the lowest validation loss for further analysis. However, this time, all models successfully lowered validation loss and increased validation accuracy for all epochs trained. The before and after results of the three most successful optimizations are shown below.

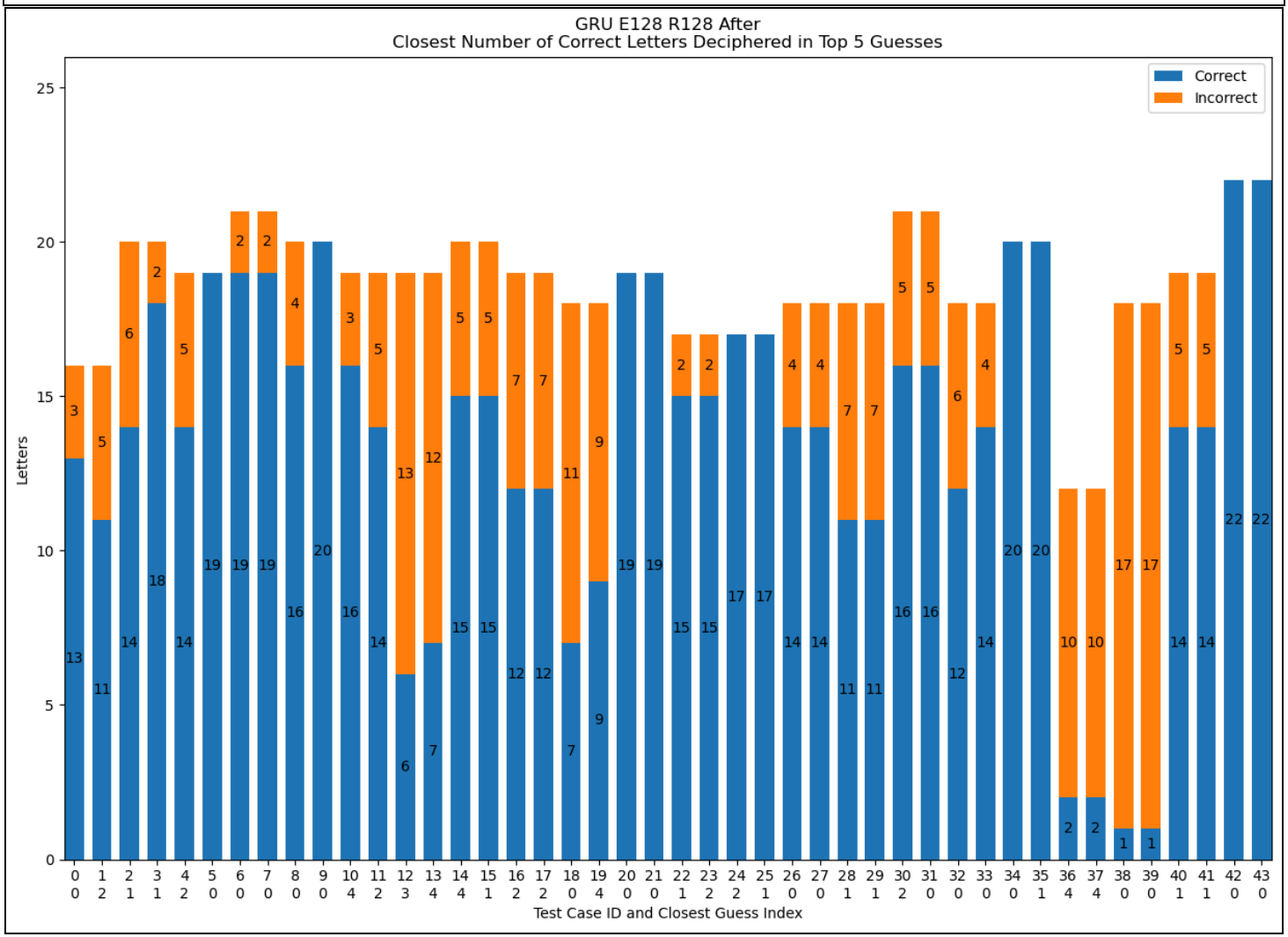
	Before	After
GRU E128 R128 Decay: 95% LR:1E-6 Epoch:10	**Training: (0.9156,0.2550)** **Validation: (0.8958,0.3248)** **Testing: (0.8930,0.3346)**	**Training: (0.9233,0.2328)** **Validation: (0.8970,0.3205)** **Testing: (0.8944,0.3301)**
LSTM E256 R128 Decay: 95% LR:5E-6 Epoch:10	**Training: (0.9261,0.2219)** **Validation: (0.8967,0.3335)** **Testing: (0.8930,0.3441)**	**Training: (0.9375,0.1905)** **Validation: (0.8987,0.3263)** **Testing: (0.8948,0.3374)**
LSTM E256 R256 Decay: 95% LR:1E-7 Epoch:10	**Training: (0.9304,0.2087)** **Validation: (0.8917,0.3278)** **Testing: (0.8944,0.3385)**	**Training: (0.9423,0.1759)** **Validation: (0.8999,0.3219)** **Testing: (0.8966,0.3327)**

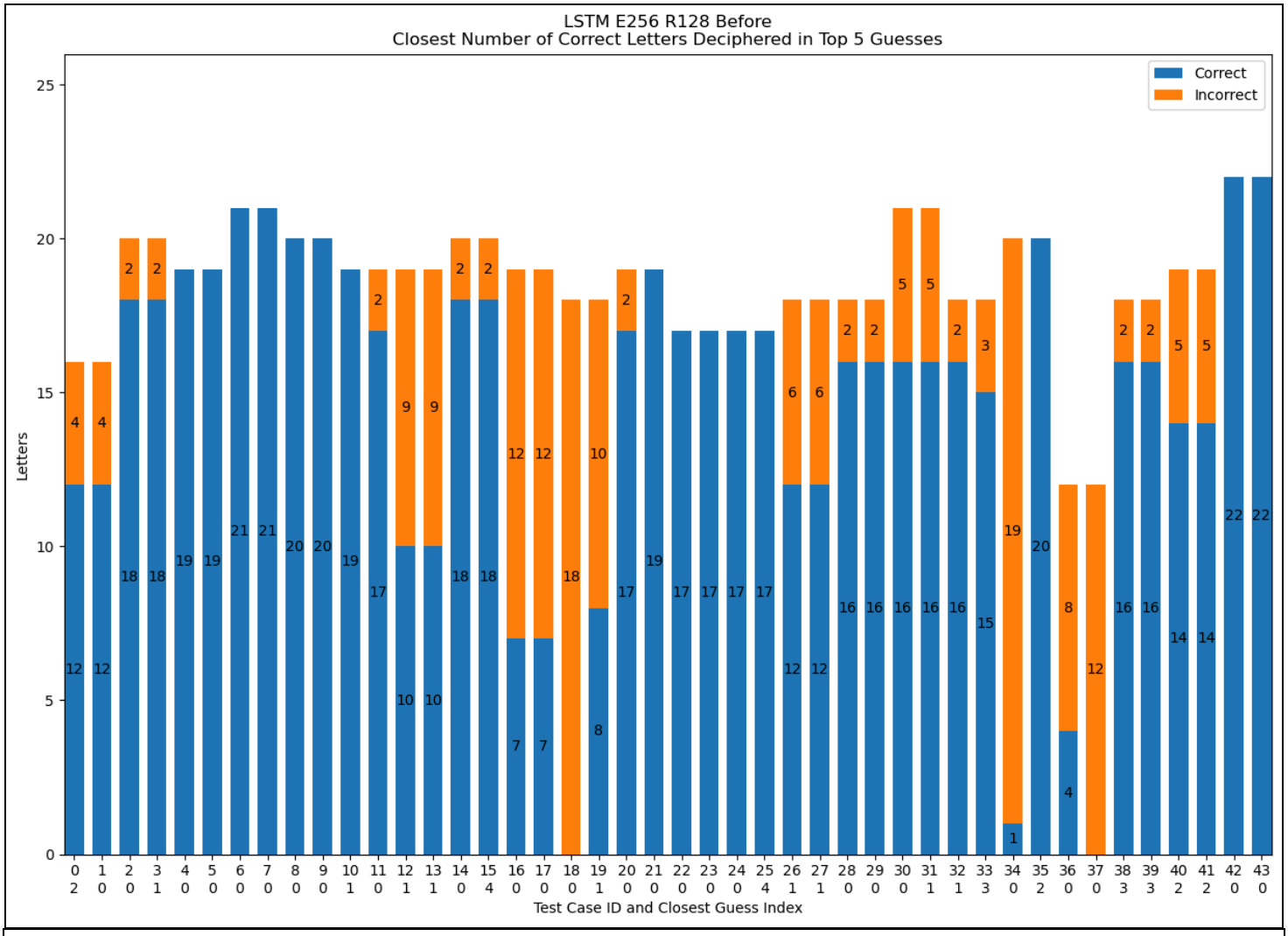
Finally, using the six models generated(before and after further optimization), had them attempt to decipher the 44 test cases from start to finish. This can be seen in the figures below. We looked at the top 5 guesses, and the closest index is noted under each guess below the test case ID. The even numbered test case IDs represent the test cases unciphered but input as all caps, and the odd numbered test case IDs decode to the same sentence as the test case ID it follows. The test cases can be found with their ID in the file final_tests.txt included in our final submission file.

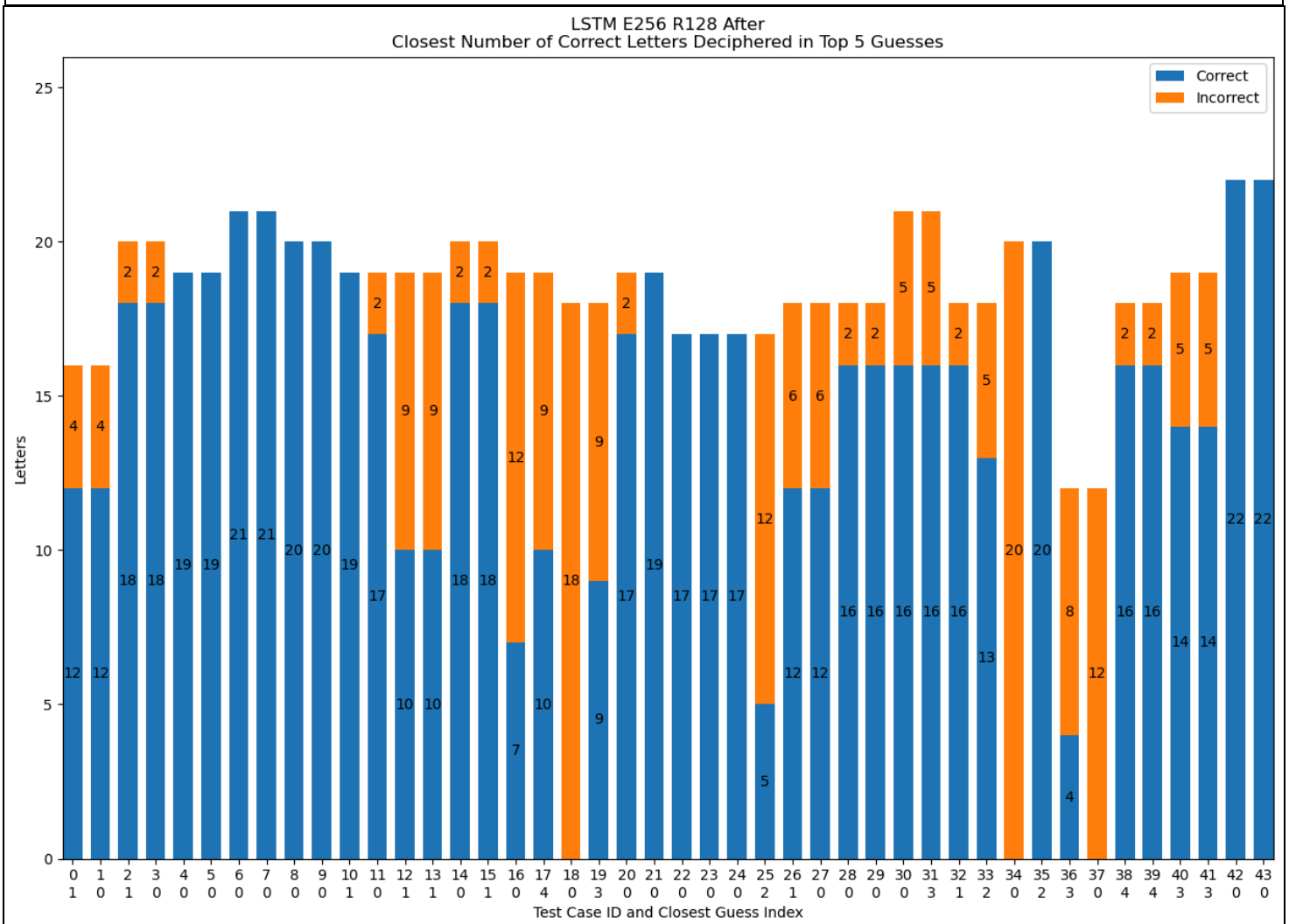
Surprisingly, it seems that the models did worse after training further. Not only is the minimum incorrect guess more wrong, but the guess index numbers seem to be more likely to be greater in the After Models when compared to the Before Models. When comparing the different models to each other, the LSTM E256 R128 Before Model is the most accurate when decoding from start to finish. It either is tied or has the most number of correct letters for each test case except for four cases, 18,34,37, and 41. The GRU Before Model has one more correct letter in case 41, and all models struggled with case 37 with the greatest success being the GRU After Model with four correct letters. What is unusual are cases 34 and 18. Both LSTM E256 R128 Models did abysmal on both of these test cases, but the other four models did pretty well on them. Even more interesting, the LSTM E256 R128 Models got all of the letters correct for its pair, the ciphered version of that sentence.

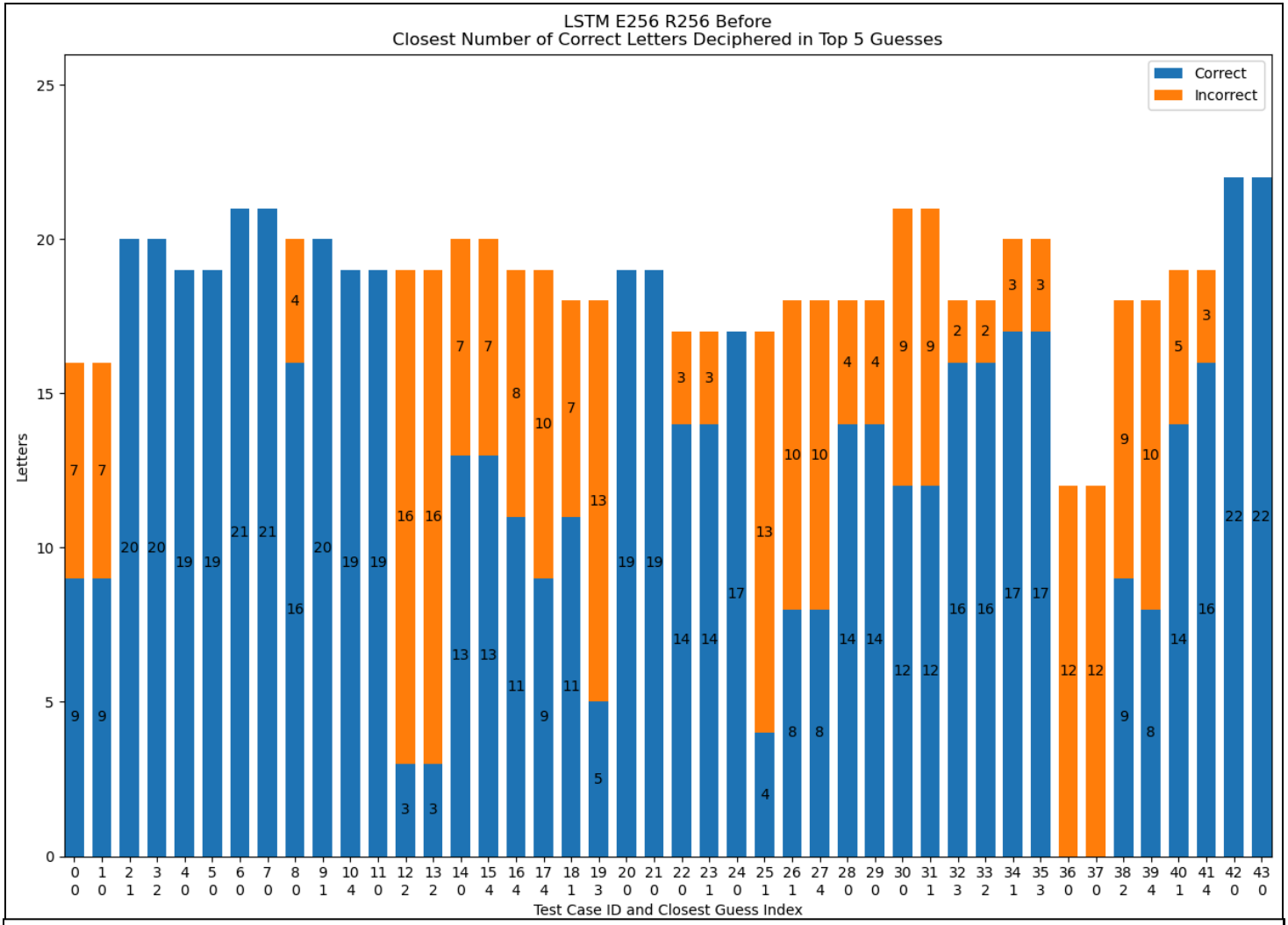
The test cases that all of the models struggled in solving were 37 and 38, which was “LIFE IS A ZOO IN A JUNGLE”, and the ciphered variant: “TMFJ MN P CGG MU P HDUWTJ”. This makes sense due two uncommon letters are used, and the length of the test case is very short. The test cases that all of the models successfully decrypted with their top guess were 42 and 43. This was the longest test case, which had the most context provided. 42 was “IT'S THE QUESTIONS WE CAN'T ANSWER THAT TEACH US THE MOST THEY TEACH US HOW TO THINK IF YOU GIVE A MAN AN ANSWER ALL HE GAINS IS A LITTLE FACT BUT GIVE HIM A QUESTION AND HE'LL LOOK FOR HIS OWN ANSWER”, and 43 a ciphered variant of it.

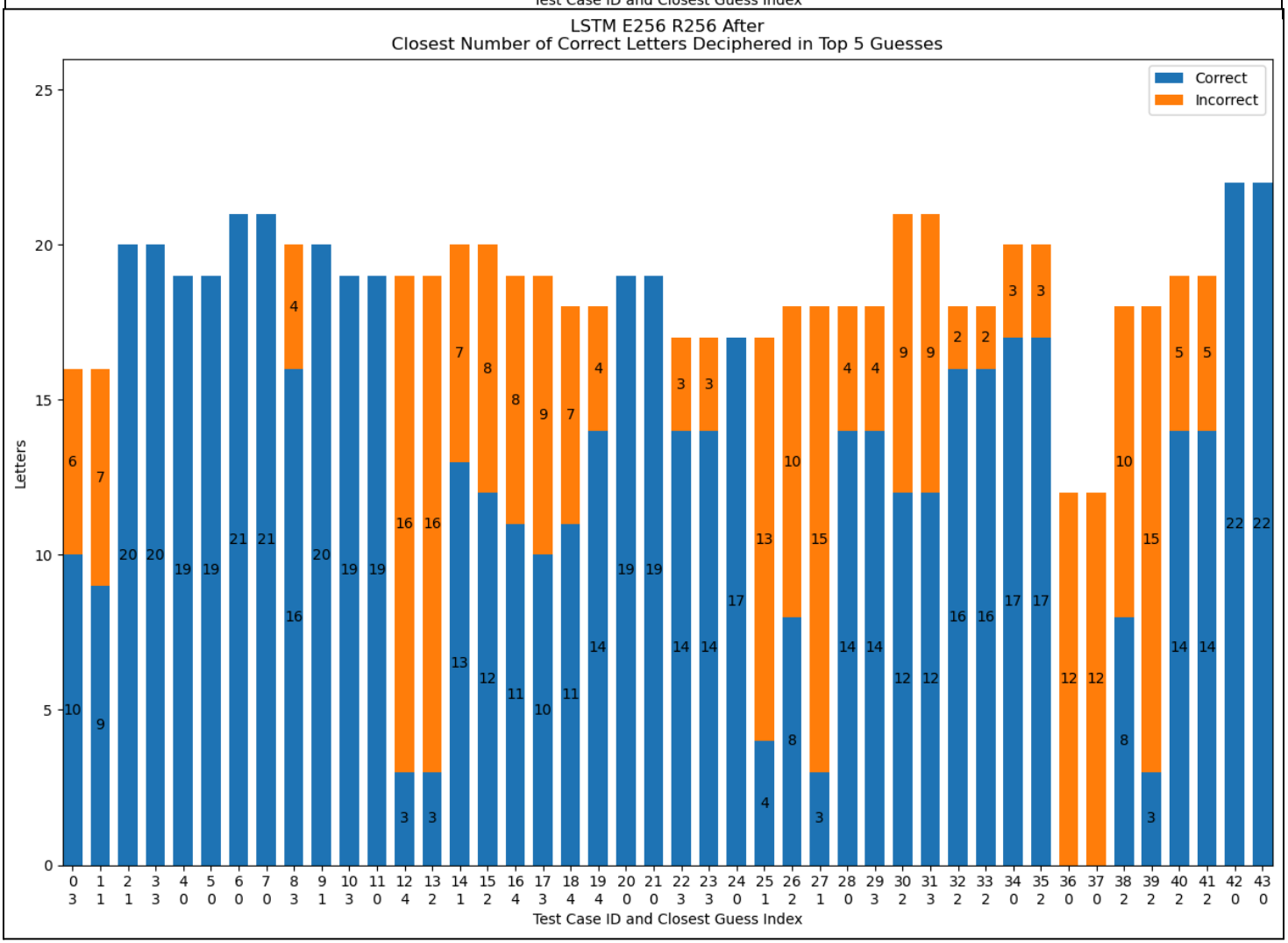

GRU E128 R128 Before
Closest Number of Correct Letters Deciphered in Top 5 Guesses
Correct
Incorrect
Letters
Test Case ID and Closest Guess Index


GRU E128 R128 After
Closest Number of Correct Letters Deciphered in Top 5 Guesses
Correct
Incorrect
Letters
Test Case ID and Closest Guess Index


LSTM E256 R128 Before
Closest Number of Correct Letters Deciphered in Top 5 Guesses
Correct
Incorrect
Letters
Test Case ID and Closest Guess Index


LSTM E256 R128 After
Closest Number of Correct Letters Deciphered in Top 5 Guesses
Correct
Incorrect
Letters
Test Case ID and Closest Guess Index


LSTM E256 R256 Before
Closest Number of Correct Letters Deciphered in Top 5 Guesses
Correct
Incorrect
Letters
Test Case ID and Closest Guess Index


LSTM E256 R256 After
Closest Number of Correct Letters Deciphered in Top 5 Guesses
Correct
Incorrect
Letters
Test Case ID and Closest Guess Index

Discussion and Conclusion

We have learned extensively about optimizing LSTM and GRU Models from this project, or more specifically, further optimizing a successful model. Even though our further trained models did worse on our curated set on fully decoding a cipher from start to finish, they did do better overall in classifying each step. If we had more time, it would be interesting to see how well the after models would do against the before models in fully deciphering simple substitution ciphers, to confirm it wasn't just our final_tests dataset that they performed poorly for.

A mistake made early on in this project happened when splitting the training, validation, and test sets. We did the split after generating the ciphers steps, so all three sets contained different steps to the same ciphers. Because of this, we were getting 97% accuracy on the training, validation, and testing sets, which was not actually representing the truth. To remedy this error, we had to retrain our models and split the sets before creating the ciphers.
Another issue that we ran into but were unable to solve was deciphering start to finish sentences that contain all letters of the alphabet. One of the sentences we tested the model on was "A quick brown fox jumped over the lazy dog". After two hours of attempting to decipher it, we ended the attempt. If one solved letter is provided to the search algorithm, the models returns results within fifteen minutes(which is still longer than normal).

To optimize this model in the future, there are a few things that we could do. One of the first things would be to increase the number of sentences that contain uncommon letters. Originally we thought it is realistic not to have as many classes that decode to uncommon letters and that would benefit our model, but after finishing analyzing the results we realized that our models were very unsuccessful at classifying them correctly. Increasing the number of sentences with zqxj doesn't make them more frequent in words, but does help our models guess them more often, and so would make the models more accurate. The next thing we could do is optimize further the order in which we were deciphering the ciphers. There are two ways to do this. The easiest to implement into our already existing code that would probably increase optimization is to look at the partially deciphered words with the same number of unique unciphered letters and choose the most common of those letters. For example, in step 11 in the corpus generation example above, there are two words with two unciphered letters. In the second word, there is an unciphered letter that is more common then the one chosen to be deciphered because the current method only looks at the first word in the word order. This method would likely increase the model accuracy. The other way to optimize the order of deciphering ciphers is significantly more complicated, but might lead to further success. We could create a new classifier model that with a given ciphered input outputs which letter is the best to classify next. The training model for this could be generated by looking in a corpus and determining which cipher letter has the least number of options to be given its surrounding context. The last thing we could optimize further is the search algorithm. Currently, the search algorithm always explores the most probable state next, but doesn't have a heuristic. By adding a heuristic, we could optimize the search to find the best sentence faster. A viable and easy to calculate heuristic to do this is the remaining number of unciphered letters. This does represent the minimum number of steps before the cipher is decoded. We did implement this and do some testing with it, however, it

ended up making our model more likely to miss uncommon letters. But, with an increase in the number of sentences that contain uncommon letters added to the training corpus, implementing the heuristic would decrease computation time in the search dramatically.

References

[1]N. Kambhatla, A. Bigvand, and A. Sarkar, “Decipherment of Substitution Ciphers with Neural Language Models,” 2018. Accessed: April 2024. [Online]. Available: https://aclanthology.org/D18-1102.pdf